

ユニット 6・第9週

シグナルと共有メモリ

📅 第9週・日程は未定 📄 演習 TD5（解答 + 生成AIチャレンジ） 📺 アニメーション 3 本

演習 TD5・解答付き

シグナルと共有メモリ（`mmap`、`shm`）

パイプを使わずにプロセスが通信する 2 つの方法：シグナルで知らせ合うことと、メモリのページを共有すること。組み合わせれば、あるプロセスがメモリにメッセージを置き、相手の「ベルを鳴らす」ことができる。

☑ 実習レポート

この実習は、授業のテンプレートを使った短い LaTeX レポートで提出します。各演習の説明、要点となるコード、端末で実行したことを書きます。LaTeX の作成には生成 AI を使えます。利用を申告し、すべての結果を確認してください。テンプレート、例、Prism のガイド →

演習問題

- | | |
|--|-------------------------|
| 1 <code>sigaction</code> で SIGUSR1 を捕捉する | 4 コードの共有：手作りの「ライブラリ」 |
| 2 <code>mmap</code> + シグナルでメッセージ | 5 System V 共有メモリ |
| 3 匿名 <code>mmap</code> の保護 | |

TD5 ・ 1

● 解答済み・テスト済み

`sigaction(2)` で SIGUSR1 を捕捉する

課題。 `mysignal` は自分の pid を表示し、シグナル番号を表示する SIGUSR1 のハンドラを設定し、シグナルを待ってから既定のハンドラに戻します。別の端末から `kill` で試してください：SIGUSR1、SIGSTOP → SIGUSR1 → SIGCONT の順、そして SIGKILL。

解答

C tds/td5/mysignal.c

```
/*
 * TD5 - exercise 1: catch SIGUSR1 with sigaction(2).
 * ./mysignal [n]      handles SIGUSR1 n times (default 3), then restores
```

```

the default action
    * From another terminal: kill -USR1 <pid>, kill -STOP <pid>, kill -CONT
    <pid>, kill -KILL <pid>
    * After the n-th SIGUSR1 the default action is back: the next SIGUSR1
    terminates the process.
    */
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

static volatile sig_atomic_t received = 0;

static void on_usr1(int sig)
{
    /* Only async-signal-safe functions here: no printf (it may be
    interrupted mid-buffer). */
    static const char msg[] = "handler: received signal ";
    char digits[4] = { (char)('0' + sig / 10), (char)('0' + sig % 10), '\n',
0 };
    if (write_all(STDERR_FILENO, msg, sizeof msg - 1) == -1 ||
write_all(STDERR_FILENO, digits, 3) == -1)
        return;
    received++;
}

int main(int argc, char *argv[])
{
    int max = argc > 1 ? atoi(argv[1]) : 3;
    struct sigaction sa = { 0 }, old;
    sigset_t block, wait_mask;

    printf("pid %d: waiting for SIGUSR1 (%d times)\n", getpid(), max);
    fflush(stdout);

    sa.sa_handler = on_usr1;
    sigemptyset(&sa.sa_mask);          /* nothing extra blocked while the
handler runs */
    sa.sa_flags = 0;
    exit_if(sigaction(SIGUSR1, &sa, &old) == -1, "sigaction");

    /* Block SIGUSR1 outside sigsuspend: a signal arriving between the test
    of `received`

```

```

        and the wait stays pending instead of being lost (the pause() race).
    */
    sigemptyset(&block);
    sigaddset(&block, SIGUSR1);
    exit_if(sigprocmask(SIG_BLOCK, &block, &wait_mask) == -1, "sigprocmask");
    sigdelset(&wait_mask, SIGUSR1);

    while (received < max)
        sigsuspend(&wait_mask);          /* atomically: unblock SIGUSR1 and
sleep */

    exit_if(sigaction(SIGUSR1, &old, NULL) == -1, "sigaction restore");
    printf("default action restored: the next SIGUSR1 terminates me\n");
    fflush(stdout);
    exit_if(sigprocmask(SIG_UNBLOCK, &block, NULL) == -1, "sigprocmask");
    for (;;)
        pause();
}

```

実際の出力

```

$ ./mysignal 2 & P=$!; sleep 0.3; kill -USR1 $P; sleep 0.2; kill -STOP $P;
kill -USR1 $P; sleep 0.2; kill -CONT $P; sleep 0.2; kill -USR1 $P; wait $P;
echo "exit status: $? (128 + 10 = killed by SIGUSR1)"
pid 357338: waiting for SIGUSR1 (2 times)
handler: received signal 10
handler: received signal 10
default action restored: the next SIGUSR1 terminates me
bash: line 1: 357338 User defined signal 1      ./mysignal 2
exit status: 138 (128 + 10 = killed by SIGUSR1)

```

- **プロセスが停止中** (SIGSTOP) でも SIGUSR1 は失われません。保留されたままで、SIGCONT のときにハンドラが実行されます。SIGSTOP と SIGKILL は捕捉もブロックもできません。管理者のための保証です。
- **ハンドラは非同期シグナル安全な関数だけを使う** (`printf` ではなく `write`) : シグナルはプログラムの `printf` の途中に届き、そのバッファに再び入り込むことがあるからです。
- `pause` の代わりに `sigprocmask + sigsuspend : received` の確認と `pause` の間にシグナルが届くと、`pause` は永遠に眠ってしまいます。シグナルをブロックしておけば保留されて待ち、`sigsuspend` がブロック解除と睡眠を1つのアトミックな手順で行います。

アニメーション A6.1・シグナルの配送 →

TD5 · 2

● 解答済み · テスト済み

mmap とシグナルによるメッセージ

課題。receiver は 128 バイトのファイルをマップし、先頭に自分の pid を書いて SIGUSR1 を待ちます。届いたらメッセージを表示します。sender は同じファイルをマップし、pid を読み、標準入力を共有領域にコピーして SIGUSR1 を送ります。128 文字より多く書くとどうなりますか？セグメントの本当の大きさはいくつですか？

解答

C tds/td5/receiver.c

```
/*
 * TD5 - exercise 2: receiver. Maps block.txt (shared), writes its pid at the
 * start of the
 * mapping, waits for SIGUSR1 and prints the message that the sender left
 * after the pid.
 * ./receiver &      then      echo "hello" | ./sender
 */
#include <fcntl.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>

#include "utils.h"

#define FILENAME "block.txt"
#define SIZE 128

static void on_usr1(int sig) { (void)sig; } /* only interrupts sigsuspend
*/

int main(void)
{
    struct sigaction sa = { 0 };
    sigset_t usr1, old, wait_mask;
```

```

    /* 1. Block SIGUSR1 BEFORE publishing the pid: a sender can only signal
    us after that,
        and the signal will wait (pending) until sigsuspend. */
    sigemptyset(&usr1);
    sigaddset(&usr1, SIGUSR1);
    exit_if(sigprocmask(SIG_BLOCK, &usr1, &old) == -1, "sigprocmask");
    sa.sa_handler = on_usr1;
    sigemptyset(&sa.sa_mask);
    exit_if(sigaction(SIGUSR1, &sa, NULL) == -1, "sigaction");

    /* 2. The file must be at least SIZE bytes: mapping past the end of a
    file gives SIGBUS. */
    int fd = open(FILENAME, O_RDWR | O_CREAT | O_TRUNC, 0644);
    exit_if(fd == -1, "open " FILENAME);
    exit_if(ftruncate(fd, SIZE) == -1, "ftruncate");
    char *shared = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd,
0);
    exit_if(shared == MAP_FAILED, "mmap");
    exit_if(close(fd) == -1, "close");          /* the mapping stays
    valid */

    /* 3. Publish our pid. */
    pid_t me = getpid();
    memcpy(shared, &me, sizeof me);
    printf("receiver %d: waiting for SIGUSR1\n", me);
    fflush(stdout);

    /* 4. Sleep with SIGUSR1 unblocked, atomically. */
    wait_mask = old;
    sigdelset(&wait_mask, SIGUSR1);
    sigsuspend(&wait_mask);
    exit_if(sigprocmask(SIG_SETMASK, &old, NULL) == -1, "sigprocmask"); /*
    SETMASK, not BLOCK */

    shared[SIZE - 1] = '\0';                    /* never trust the
    other side */
    printf("received: %s", shared + sizeof me);
    exit_if(munmap(shared, SIZE) == -1, "munmap");
    return EXIT_SUCCESS;
}

```

C tds/td5/sender.c

```
int main(void)
{
    int fd = open(FILENAME, O_RDWR);
    exit_if(fd == -1, "open " FILENAME);
    char *shared = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd,
0);
    exit_if(shared == MAP_FAILED, "mmap");
    exit_if(close(fd) == -1, "close");

    pid_t pid;
    memcpy(&pid, shared, sizeof pid);
    printf("sender: the receiver is %d\n", pid);

    /* Copy stdin into the message area, keeping one byte for '\0'. */
    size_t off = sizeof pid, max = SIZE - 1;
    ssize_t n = 1;
    while (off < max && (n = read(STDIN_FILENO, shared + off, max - off)) >
0)
        off += (size_t)n;
    exit_if(n == -1, "read");
    shared[off] = '\0';

    exit_if(msync(shared, SIZE, MS_SYNC) == -1, "msync"); /* optional: also
push to the file */
    exit_if(kill(pid, SIGUSR1) == -1, "kill");
    exit_if(munmap(shared, SIZE) == -1, "munmap");
    return EXIT_SUCCESS;
}
```

実際の出力

```
$ ./receiver & sleep 0.3; echo 'hello from the sender' | ./sender; wait
receiver 357391: waiting for SIGUSR1
sender: the receiver is 357391
received: hello from the sender
```

C tds/td5/mmap_limits.c

```
/*
 * TD5 - exercise 2, last question: how big is the shared segment of a 128-
byte file?
 * The kernel maps whole pages. We map TWO pages of a 128-byte file and write
byte by byte:
```

```

*   bytes 0..127      belong to the file
*   bytes 128..4095   exist (same page) but are never written back to the
file
*   byte 4096         is in a page that lies entirely past the end of the
file: SIGBUS
* The signal is caught and siglongjmp brings us back to report where it
happened.
*/
#include <fcntl.h>
#include <setjmp.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <unistd.h>

#include "utils.h"

#define FILENAME "limits.txt"
#define FILESIZE 128

static sigjmp_buf env;
static volatile sig_atomic_t caught;
static char *volatile map;          /* volatile: must survive the
siglongjmp */
static volatile long offset;

static void on_fault(int sig)
{
    caught = sig;
    siglongjmp(env, 1);
}

int main(void)
{
    const long page = sysconf(_SC_PAGESIZE);
    int fd = open(FILENAME, O_RDWR | O_CREAT | O_TRUNC, 0644);
    exit_if(fd == -1, "open");
    exit_if(ftruncate(fd, FILESIZE) == -1, "ftruncate");
    map = mmap(NULL, 2 * page, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    exit_if(map == MAP_FAILED, "mmap");

    struct sigaction sa = { 0 };
    sa.sa_handler = on_fault;
    sigemptyset(&sa.sa_mask);
  
```

```

    exit_if(sigaction(SIGSEGV, &sa, NULL) == -1 || sigaction(SIGBUS, &sa,
NULL) == -1, "sigaction");

    if (sigsetjmp(env, 1) == 0)
        for (offset = 0; offset < 2 * page; offset++)
            map[offset] = 'x';

    printf("file size %d bytes, page size %ld bytes, mapping of %ld bytes\n",
FILESIZE, page, 2 * page);
    printf("wrote bytes 0..%ld; byte %ld raised %s\n", offset - 1, offset,
caught ? signame(caught) : "nothing");

    exit_if(munmap(map, 2 * page) == -1, "munmap");
    off_t end = lseek(fd, 0, SEEK_END);          /* the file did not
grow */
    exit_if(end == -1, "lseek");
    printf("file size after the writes: %lld bytes\n", (long long)end);
    exit_if(close(fd) == -1, "close");
    return EXIT_SUCCESS;
}

```

実際の出力

```

$ ./mmap_limits
file size 128 bytes, page size 4096 bytes, mapping of 8192 bytes
wrote bytes 0..4095; byte 4096 raised SIGBUS
file size after the writes: 128 bytes

```

本当の大きさは1ページ（4096バイト）です。カーネルはページ単位でマップします。128～4095 バイト目はメモリには存在しますが、ファイルのものではありません（ディスクには書かれず、ファイルは128 バイトのままです）。ファイルの終わりより完全に先にあるページに触れると **SIGBUS** になります。だから `receiver` は `ftruncate` を呼びます。空のファイルをマップして書き込んでも SIGBUS になります。

⚠ 修正版

引き継いだ版は `SIG_BLOCK` でマスクを戻そうとしていた（戻すのではなく追加していた）、ハンドラの中で `printf` を呼んでいた、そしてメッセージが公開した pid を上書きしていました。今は pid が先頭のバイトを占め、メッセージはその後続きます。

TD5 ・ 3

● 解答済み・実際の出力

匿名 mmap の保護

課題。 `map_anonymous` は

`mmap(NULL, sizeof(int), PROT, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0)` で整数1つ分を求め、ポインタを表示し、`*p` を読み、42 を書いてもう一度読みます。 `PROT_NONE`、`PROT_READ`、`PROT_WRITE`、`PROT_EXEC`、`PROT_EXEC|PROT_WRITE` でビルドし、それぞれの結果を `mmap(2)` で説明してください。

解答



C tds/td5/map_anonymous.c

```

/*
 * TD5 - exercise 3: protections of an anonymous mapping.
 * PROT is given at compile time; the Makefile builds map_none, map_read,
map_write, map_exec
 * and map_ew (PROT_EXEC|PROT_WRITE). Each program reads *p, writes 42 and
reads it again.
 * Read mmap(2): on x86-64, PROT_WRITE implies read access; PROT_NONE forbids
everything.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>

#include "utils.h"

#ifndef PROT
#define PROT (PROT_READ | PROT_WRITE)
#endif

/* Readable name of the protection flags, e.g. "PROT_EXEC|PROT_WRITE". */
static const char *prot_name(int prot)
{
    static char name[64];
    name[0] = '\0';
    if (prot == PROT_NONE)
        return "PROT_NONE";
    if (prot & PROT_READ) strcat(name, "|PROT_READ");
    if (prot & PROT_WRITE) strcat(name, "|PROT_WRITE");
    if (prot & PROT_EXEC) strcat(name, "|PROT_EXEC");
    return name + 1;
}

int main(void)
{
    int *p = mmap(NULL, sizeof(int), PROT, MAP_PRIVATE | MAP_ANONYMOUS, -1,
0);
    exit_if(p == MAP_FAILED, "mmap");

    fprintf(stderr, "PROT = %s\n", prot_name(PROT));
    fprintf(stderr, "pointer: %p\n", (void *)p);
    fprintf(stderr, "read:    *p = %d\n", *p);          /* anonymous memory

```

```
starts zeroed */
    *p = 42;
    fprintf(stderr, "write:  *p = %d\n", *p);

    exit_if(munmap(p, sizeof(int)) == -1, "munmap");
    return EXIT_SUCCESS;
}
```

実際の出力

```
$ ./map_none
PROT = PROT_NONE
pointer: 0x742c16995000
bash: line 1: 357416 Segmentation fault      (core dumped) ./map_none
[exit status 139]
```

実際の出力

```
$ ./map_read
PROT = PROT_READ
pointer: 0x7b6b0d393000
read:  *p = 0
bash: line 1: 357431 Segmentation fault      (core dumped) ./map_read
[exit status 139]
```

実際の出力

```
$ ./map_write
PROT = PROT_WRITE
pointer: 0x7c7095214000
read:  *p = 0
write: *p = 42
```

実際の出力

```
$ ./map_exec
PROT = PROT_EXEC
pointer: 0x75b3a49d8000
bash: line 1: 357495 Segmentation fault      (core dumped) ./map_exec
[exit status 139]
```

実際の出力

```
$ ./map_ew
PROT = PROT_WRITE|PROT_EXEC
pointer: 0x7872e8140000
read:    *p = 0
write:   *p = 42
```

MMU は各ページの権限を強制します：`PROT_NONE` は読むことさえ禁止、`PROT_READ` は読めますが（匿名メモリはゼロから始まります）書けません。x86-64 では `PROT_WRITE` は読み込みも意味します。`PROT_EXEC` だけの場合は興味深いケースです：このマシン（メモリ保護キーあり）では Linux が実行専用メモリを作るので読み込みは失敗し、その機能のないプロセッサでは同じテストで 0 が読めます。だから `mmap(2)` は各組み合わせの効果は「アーキテクチャ依存」だと書いています。違反はどれもページフォールトで、カーネルがそれを SIGSEGV（ステータス 139）に変えます。

TD5 ・ 4

● 解答済み・テスト済み

コードの共有：手作りの「ライブラリ」

課題。共有ライブラリをまねます：`load_add` は `add(int, int)` の機械語を、`MAP_SHARED` でマップした 2048 バイトのファイルにコピーします。`use_add` はそのファイルを `PROT_EXEC` でマップし、関数ポインタとして使って `f(42, 12)` を呼びます。`use_add` を同時にいくつも動かすとどうなりますか？

解答



C tds/td5/load_add.c

```
int add(int a, int b)
{
    return a + b;
}

int main(void)
{
    int fd = open(LIBFILE, O_RDWR | O_CREAT | O_TRUNC, 0644);
    exit_if(fd == -1, "open " LIBFILE);
    /* An empty file cannot be mapped usefully (access past its end =
    SIGBUS): give it its size. */
    exit_if(ftruncate(fd, SIZE) == -1, "ftruncate");

    char *code = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    exit_if(code == MAP_FAILED, "mmap");
    exit_if(close(fd) == -1, "close");

    /* Copy the first bytes of add(). Converting a function pointer to void *
    is not ISO C,
       but POSIX requires it to work (dlsym relies on it). */
    memcpy(code, (const void *)add, 64);

    exit_if(munmap(code, SIZE) == -1, "munmap"); /* MAP_SHARED: the bytes
    are in the file */
    return EXIT_SUCCESS;
}
```

C tds/td5/use_add.c

```
int main(void)
{
    int fd = open(LIBFILE, O_RDONLY);
    exit_if(fd == -1, "open " LIBFILE);

    void *code = mmap(NULL, SIZE, PROT_READ | PROT_EXEC, MAP_SHARED, fd, 0);
    exit_if(code == MAP_FAILED, "mmap");
    exit_if(close(fd) == -1, "close");

    int (*f)(int, int) = (int (*)(int, int))code;
    printf("f(42, 12) = %d\n", f(42, 12));

    exit_if(munmap(code, SIZE) == -1, "munmap");
    return EXIT_SUCCESS;
}
```

実際の出力

```
$ ./load_add && ./use_add
f(42, 12) = 54
```

動くのは、`add` が絶対アドレスを使わない位置独立なコードだからです。複数の `use_add` は同じ物理ページ（ファイルのページキャッシュのページ）を共有します。すべてのプログラムが `libc.so` のページを共有するのと同じです。本物のローダ（`ld.so`、`dlopen`）は、さらにシンボルと再配置を解決します。引き継いだ版はファイルに大きさを与えずにマップしており、SIGBUS で終わっていました。

TD5 ・ 5

● 解答済み・テスト済み

System V 共有メモリ

課題。(1) `ftok` + `shmget` で 1024 バイトのセグメントを作るか取得し、`shmat` で取り付け、最初の整数を 1 増やして表示し、`shmdt` で外してください。何度か実行すると何が見えますか？ (2) 4 回まわるループの中で `fork` を使い、16 個のプロセスが自分の pid をセグメントに登録し (pid の数 + pid の配列)、最初のプロセスが一覧を表示します。32 プロセスでも繰り返してください。

解答



C tds/td5/shm_counter.c

```

/*
 * TD5 - exercise 5: a System V shared memory segment that survives the
 * processes.
 * ./shm_counter increments the first int of the segment and
 * prints it
 * ./shm_counter --remove destroys the segment (IPC_RMID)
 * Run it several times: 1, 2, 3... The segment lives in the kernel until it
 * is removed or
 * the machine reboots (list them with `ipcs -m`, remove with `ipcrm`).
 * The key comes from ftok(file, id): same file + same id = same key on this
 * machine.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>

#include "utils.h"

#define KEYFILE "/etc/passwd" /* any file that exists on every Linux
machine */
#define KEYID 'F'
#define SIZE 1024

int main(int argc, char *argv[])
{
    key_t key = ftok(KEYFILE, KEYID);
    exit_if(key == -1, "ftok");
    int id = shmget(key, SIZE, IPC_CREAT | 0644);
    exit_if(id == -1, "shmget");

    if (argc > 1 && strcmp(argv[1], "--remove") == 0) {
        exit_if(shmctl(id, IPC_RMID, NULL) == -1, "shmctl IPC_RMID");
        printf("segment %d removed\n", id);
        return EXIT_SUCCESS;
    }

    int *counter = shmat(id, NULL, 0);
    exit_if(counter == (void *)-1, "shmat");
    counter[0]++; /* a fresh segment is zero-
filled */

```

```
printf("segment %d (key 0x%08x): counter = %d\n", id, (unsigned)key,
counter[0]);
exit_if(shmdt(counter) == -1, "shmdt");
return EXIT_SUCCESS;
}
```

実際の出力

```
$ ./shm_counter; ./shm_counter; ./shm_counter; ./shm_counter --remove
segment 65593 (key 0x4605476d): counter = 1
segment 65593 (key 0x4605476d): counter = 2
segment 65593 (key 0x4605476d): counter = 3
segment 65593 removed
```

C tds/td5/shm_fork.c

```
/*
 * TD5 - exercise 5 (end): 16 processes (4 fork() in a loop) register their
 * pid in a shared
 * segment: segment[0] = number of pids, then the pids. The original process
 * prints the list.
 *
 * "count++; table[count] = pid" is NOT atomic: two processes can read the
 * same count and
 * one registration is lost. Build with -DNO_ATOMIC (Makefile target
 * shm_race) to see it;
 * the default version reserves the slot with an atomic fetch-and-add (a
 * preview of unit 7).
 * The segment is IPC_PRIVATE: only this process and its descendants (which
 * inherit the
 * attachment through fork) can see it.
 */
#include <sched.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

#define FORKS 4 /* 2^4 = 16 processes */
#define SIZE 4096
```

```
int main(int argc, char *argv[])
{
    int forks = argc > 1 ? atoi(argv[1]) : FORKS;
    int id = shmget(IPC_PRIVATE, SIZE, IPC_CREAT | 0600);
    exit_if(id == -1, "shmget");
    int *seg = shmat(id, NULL, 0);
    exit_if(seg == (void *)-1, "shmat");
    /* Mark for deletion now: it disappears when the last process detaches
    (even on a crash). */
    exit_if(shmctl(id, IPC_RMID, NULL) == -1, "shmctl");

    pid_t original = getpid();
    int children = 0;
    for (int i = 0; i < forks; i++) {
        pid_t pid = fork();
        exit_if(pid == -1, "fork");
        if (pid == 0)
            children = 0;          /* the child starts with no children
of its own */
        else
            children++;
    }

#ifdef NO_ATOMIC
    int slot = seg[0] + 1;          /* read ... */
    sched_yield();                 /* ... (widen the window) ... */
    seg[0] = slot;                 /* ... write: not atomic */
#else
    int slot = __atomic_add_fetch(&seg[0], 1, __ATOMIC_SEQ_CST); /* read-
modify-write in one step */
#endif
    if (slot < SIZE / (int)sizeof(int))
        seg[slot] = getpid();

    while (children-- > 0)
        exit_if(wait(NULL) == -1, "wait");

    if (getpid() == original) {
        printf("%d processes registered (expected %d):", seg[0], 1 << forks);
        for (int i = 1; i <= seg[0] && i < SIZE / (int)sizeof(int); i++)
            printf(" %d", seg[i]);
        printf("\n");
    }
    exit_if(shmdt(seg) == -1, "shmdt");
}
```

```
return EXIT_SUCCESS;
}
```

実際の出力

```
$ ./shm_fork
16 processes registered (expected 16): 357564 357566 357567 357572 357568
357571 357569 357565 357574 357573 357570 357576 357578 357575 357577 357579
```

実際の出力

```
$ for i in 1 2 3 4 5; do ./shm_race 5 | cut -d: -f1; done
19 processes registered (expected 32)
28 processes registered (expected 32)
25 processes registered (expected 32)
23 processes registered (expected 32)
27 processes registered (expected 32)
```

- **セグメントはプロセスより長く生きる**：カウンタは1、2、3...と増えます。誰かが削除する（`IPC_RMID`、`ipcrm`）かマシンを再起動するまで、セグメントはカーネルの中に残るからです。`ipcs -m`で見えます。引き継いだ版は最初の実行で削除していたので、この現象が見えませんでした。
- **32 プロセスでは登録が失われる**（上の5回の実測では32個中16～25個）：`seg[0]++`は「読む・足す・書く」なので、2つのプロセスが同じ値を読むことがあります。これが競合状態で、ユニット7のテーマです。解決策はプロセッサのアトミックな加算（`__atomic_add_fetch`）を使うことです。
- `ftok`にはどのLinuxにもあるファイル（`/etc/passwd`）を使います。元の課題文はUbuntuには存在しない`/etc/bashrc`を使っていました。

チェックリスト

生成AIチャレンジ・`pause`を`sigsuspend`に置き換える...そして競合を証明する



目標。 AIと一緒に、`pause()`でシグナルを失うプログラム（古典的な競合）を書き、それがどれくらいの頻度で起きるかを測り、`sigprocmask` + `sigsuspend` でなくなることを示す。

最初のプロンプト。 好きなツール（Claude、ChatGPT、Gemini、Copilot、Codex...）で、演習の解答付きバージョンをコンテキストとして使ってください。まず計画を求め、各ステップを確認してから受け入れましょう。

Prompt

`pause()` の競合を実験で示したいです。次を書くのを手伝ってください：(1) `arrived = 1` にするハンドラを持ち、`while (!arrived) pause();` を行う受け手。(2) 受け手が `pid` を公開した直後に `SIGUSR1` を送る送り手。(3) 1 秒の制限付きで実験を 1000 回繰り返し、受け手が永遠に眠った回数を数えるスクリプト。その後、`sigprocmask` + `sigsuspend` を使う版を書いて比べましょう。シグナルが失われる正確な時間の窓を説明してください。

結果の確かめ方

- `pause()` 版は少なくとも何回かは固まる（一度も固まらなければ、`pause` の前に `usleep` を入れて窓を広げる）。
- `sigsuspend` 版は 1000 回実行して一度も固まらない。
- 確認と待機の間で正確な行を指し示せる。

AI の利用を申告しましょう：使ったツール、主なプロンプト、自分で確認したこと、修正したこと。大切なのは、すべての判断を説明し検証できることです。